CS 4400: Artificial Intelligence

First-order logic 2

Contents

1. Inference by propositionalization
2. New inference rules
   - Unification
   - Generalized Modus Ponens
   - Resolution
3. Forward- and backward-chaining

1/3. Inference in FOL

- In propositional logic, we had several inference rules
  - Modus Ponens, Modus Tolens, etc.
  - Resolution (single rule)
- Good news: These apply to FOL!
- Bad news: Need to figure out how to deal with quantifiers

Universal Instantiation

- We are allowed to substitute any term for a universally quantified variable

  $\forall x \alpha$

  SUBST({x/g},α)

  where SUBST({x/g}, α) is the result of substituting g for x in sentence α

- Example:
  - $\forall x$Duck(x) ⇒Bird(x)

    : Duck(Howard) ⇒ Bird(Howard) {x/Howard}

    : Duck(G.Washington) ⇒Bird(G.Washington) {x/GW}

Existential Instantiation

- For an existential quantifier, we are allowed to introduce <u>one new constant</u> into our KB

  $\frac{\exists x\ \alpha}{\text{SUBST}(\{x/k\}, \alpha)}$

- Example:
  - $\exists x$ Duck(x)

    : Duck(Howard123) {x/Howard123}
  - $\exists x$ Crown(x) $\wedge$OnHead(x,John)

    : Crown(C_1) $\wedge$ OnHead(C_1,John) {x/C_1}

Existential Instantiation

- Universal Instantiation can be applied many times to infer different things
- Existential Instantiation need only be applied once
- Existential instantiation is also called *Skolemization*
  - The constant is known as a *Skolem* constant

Example

- KB
  - Duck(Howard) — given
  - ∀xDuck(x) ⇒Bird(x) — given
- Prove Howard is a Bird

- Proof:
  - Duck(Howard) ⇒Bird(Howard) — UI{x/Howard}
  - Bird(Howard) — Modus Ponens

Propositionalization

- We can “propositionalize” the FOL sentences by instantiation, and use propositional inference rules
- A problem: functions can generate an infinite number of sentences
  - Duck(Howard) ⇒Bird(Howard)
  - Duck(FatherOf(Howard)) ⇒Bird(FatherOf(Howard))
  - Duck(FatherOf(FatherOf(Howard))) ⇒Bird(FatherOf(FatherOf(Howard)))
- Herbrand(1930): we only need a finite depth of nesting to prove entailments
  - When the query is not entailed, nesting doesn’t stop….
- The question of entailment for first-order logic is semidecidable

Reduction to propositional form

Suppose the KB contains the following:

$\forall x$ King(x) $\wedge$ Greedy(x) $\Rightarrow$ Evil(x)
King(John)
Greedy(John)
Brother(Richard,John)

- Instantiating the universal sentence in all possible ways, we have: (there are only two ground terms: John and Richard)

King(John) $\wedge$ Greedy(John) $\Rightarrow$ Evil(John)
King(Richard) $\wedge$ Greedy(Richard) $\Rightarrow$ Evil(Richard)
King(John)
Greedy(John)
Brother(Richard,John)

- The new KB is propositionalized with "propositions":

King(John), Greedy(John), Evil(John), King(Richard), etc.

Reduction continued

- Every FOL KB can be propositionalized so as to preserve entailment
  - A ground sentence is entailed by new KB iff entailed by original KB

- Idea for doing inference in FOL:
  - propositionalize KB and query
  - apply resolution-based inference
  - return result

- Problem: with function symbols, there are infinitely many ground terms,
  - e.g., *Father*(*Father*(*Father*(*John*))), etc

Reduction continued

Theorem: Herbrand (1930). If a sentence α is entailed by a FOL KB, it is entailed by a finite subset of the propositionalized KB

Idea: For n = 0 to ∞ do
create a propositional KB by instantiating with depth n terms
see if α is entailed by this KB

Problem: works if α is entailed, loops if α is not entailed.
→ The problem of semi-decidable: algorithms exist to prove entailment, but no algorithm exists to prove non-entailment for every non-entailed sentence.

Other Problems with Propositionalization

- Propositionalization generates lots of irrelevant sentences
  - So inference may be very inefficient
- e.g., from:

  $\forall x$ King(x) $\wedge$ Greedy(x) $\Rightarrow$ Evil(x)
  King(John)
  $\forall y$ Greedy(y)
  Brother(Richard,John)

- it seems obvious that *Evil*(*John*) is entailed, but propositionalization produces lots of facts such as *Greedy*(*Richard*) that are irrelevant
- With p k-ary predicates and n constants, there are $p \cdot n^k$ instantiations
- Lets see if we can do inference directly with FOL sentences

2/3. New inference rules

- Unification
- Generalized Modus Ponens
- Generalized Resolution

Unification

- Unification is the process of finding substitutions to make two sentences the same on demand
  - If SUBST({x/z},p) = SUBST({y/z},q), then
  - UNIFY(p,q) = {x/z, y/z} : common constant/variable that makes two sentences the same after substitution
- Example:
  - UNIFY(Duck(y),Duck(Howard)) = {y/Howard}
  - UNIFY(Duck(Donald),Duck(Howard)) = FAIL

More examples

- KB:
  - Knows(John,James)
  - Loves(John,Mary)
  - Knows(John,Mary)
  - ∀xKnows(Fred,x)
- Query
  - ∃x Knows(x,Mary)

- KB:
  - Knows(John,James)
  - Loves(John,Mary)
  - Knows(John,Mary)
  - ∀xKnows(Fred,x)
- Query
  - ∃x Knows(x,Mary)

- Ignore quantifiers
- Predicate names must match
- Proceed stepwise, left to right
- Constants match exact constants or a variable
- Variable can be substituted for another variable

- KB:
  - Knows(John,James)
  - Loves(John,Mary)
  - Knows(John,Mary)
  - ∀xKnows(Fred,x)
- Query
  - ∃x Knows(x,Mary)

- Ignore quantifiers
- Predicate names must match
- Proceed stepwise, left to right
- Constants match exact constants or a variable
- Variable can be substituted for another variable

1. UNIFY(Knows(x,Mary), Knows(John,James)) = FAIL
2. UNIFY(Knows(x,Mary), Loves(John,Mary)) = FAIL
3. UNIFY(Knows(x,Mary), Knows(John,Mary)) = {x/John}
4. UNIFY(Knows(Fred,x), Knows(x,Mary)) = FAIL

- UNIFY(Knows(Fred,x), Knows(x,Mary)) = FAIL, because *x* cannot take on two values at once
- A simple fix to solve this problem: “standardizing apart”
  - Rewrite variables with the same name but in different sentences to be different “on the fly” for unification
    - UNIFY(Knows(Fred,x), Knows(x99,Mary)) = {x/Mary, x99/Fred}
  - Make sure you keep variables in a sentence consistent!
    - ∀xKnows(Fred,x) ⇒Hates(Fred,x)
      - : ∀x99Knows(Fred,x99) ⇒Hates(Fred,x99) – Good!
      - : ∀xKnows(Fred,x99) ⇒Hates(Fred,x) – Bad!

One more examples

- simple example: query = Knows(John,x), i.e., who does John know?

p	q	θ
Knows(John,x)	Knows(John,Jane)	{x/Jane}
Knows(John,x)	Knows(y,OJ)	{x/OJ,y/John}
Knows(John,x)	Knows(y,Mother(y))	{y/John,x/Mother(John)}
Knows(John,x)	Knows(x,OJ)	{fail}

- Last unification fails: only because x can't take values John and OJ at the same time
  - But we know that if John knows x, and everyone (x) knows OJ, we should be able to infer that John knows OJ
- Problem is due to use of same variable x in both sentences
- Simple solution: Standardizing apart eliminates overlap of variables, e.g., Knows(z,OJ)

Generalized Modus Ponens

- Generalized Modus Ponens in PL

  $\underline{A \Rightarrow B,\ A}$

  B
- Generalized Modus Ponens in FOL

  $\underline{p1 \wedge p2 \wedge \ldots \wedge pn \Rightarrow B,\quad q1, q2, \ldots, qn}$

  SUBST({ … /x},B)

  where for each pair of sentences (pi, qi),
  1. pi and qi have the same predicates
  2. UNIFY(pi, qi) is successful

Generalized Modus Ponens

- KB:
  1. $\forall x \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
  2. $\text{King}(\text{John})$
  3. $\forall y \, \text{Greedy}(y)$

- Query: Evil(John)?

- If you can UNIFY the symbols in the *antecedent* with other facts …
- … then you can conclude the *consequent* with UNIFICATION applied

Generalized Modus Ponens

- KB:
  1. $\forall x \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
  2. $\text{King}(\text{John})$
  3. $\forall y \text{Greedy}(y)$
  4. $\forall y \text{King}(\text{John}) \wedge \text{Greedy}(y)$ 2,3 And Introduction
  5. $\text{Evil}(\text{John})$ 1,4, GMP, $\{x/\text{John}, y/\text{John}\}$

- Query: Evil(John)?

- If you can UNIFY the symbols in the *antecedent* with other facts …
- … then you can conclude the *consequent* with UNIFICATION applied

Generalized Modus Ponens

- KB:
  1. $\forall x \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
  2. $\exists y \text{ King}(y) \wedge \text{Greedy}(y)$

- Query: $\exists z \text{ Evil}(z)$?

Generalized Modus Ponens

- KB:
  1. $\forall x \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
  2. $\exists y \text{ King}(y) \wedge \text{Greedy}(y)$
  3. $\text{King}(\text{Person99}) \wedge \text{Greedy}(\text{Person99})$ 2, EI {y/Person99}
  4. $\text{Evil}(\text{Person99})$ 1, 3, GMP {x/Person99}

- Query: $\exists z \text{ Evil}(z)$?
- 2 tells us some person who is a King and is Greedy exists
- Use EI to introduce a symbol for this person (i.e. “Person99”)
- Use GMP on 1 and 3 to conclude that Person99 is Evil

Generalized Resolution

- Resolution in PL:

  $\underline{A \vee B \vee \ldots \vee F \vee G, \quad \sim G \vee H \vee \ldots \vee Z}$

  $A \vee B \vee \ldots \vee F \vee H \vee \ldots \vee Z$

- Generalized resolution in FOL:

  $\underline{A \vee B \vee \ldots \vee F \vee G, \quad \sim G' \vee H \vee \ldots \vee Z}$

  SUBST({…/x}, A v B v … v F v H v … v Z)

  where UNIFY(G,G') = x

Generalized Resolution

- KB:
  1. $\forall x,y\ Sibling(x,y) \wedge Male(x) \Rightarrow Brother(x,y)$
  2. $\forall x,y\ Sibling(x,y) \wedge \sim Male(x) \Rightarrow Sister(x,y)$
  3. $\forall x,y\ Sibling(x,y) \Rightarrow Sibling(y,x)$
  4. $Male(John)$
  5. $\sim Male(Mary)$
  6. $Sibling(Mary,John)$

- Query: Brother(John,Mary) ?

Generalized Resolution

Change KB to CNF

1.	∀x,y ~Sibling(x,y) ∨~Male(x) ∨Brother(x,y)		given (1 –Impl. Elim.)
2.	∀x,y ~Sibling(x,y) ∨Male(x) ∨Sister(x,y)		given (2 –IE)
3.	∀x,y ~Sibling(x,y) ∨Sibling(y,x)		given (3 –IE)
4.	Male(John)		given
5.	~Male(Mary)		given
6.	Sibling(Mary,John)		given

Introduce ~conclusion

7.	~Brother(John,Mary)		ASSUMPTION
8.	Sibling(John,Mary)	3,6, UR,	{x/Mary, y/John}
9.	~Male(John) ∨Brother(John,Mary)	1,8, UR,	{x/John, y/Mary}
10.	Brother(John,Mary)	4,9, UR	
11.	CONTRADICTION !		7,10

3/3. Forward and backward chaining

- Similar to forward/backward chaining in propositional logic
- Example: Colonel West domain
  - KB

    “The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.”
  - Query: Is Colonel West a criminal?

Colonel West domain

- Constants: Nono, West, America
- Predicates:
  - Unary: American(x), Weapon(x), Missile(x), Hostile(x), Criminal(x)
  - Binary: Owns(x,y), Enemy(x,y)
  - Tertiary: Sells(x,y,z)

1. Implicit knowledge - missiles are weapons:
2. Implicit knowledge - an enemy of America counts as "hostile":
3. "West, who is American . . .":
4. "The country Nono, an enemy of America . . .":

- Missiles are weapons:
  - $\forall x \text{Missile}(x) \Rightarrow \text{Weapon}(x)$
- An enemy of America counts as “hostile”:
  - $\forall x \text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$
- “West, who is American . . .”:
  - $\text{American}(\text{West})$
- “The country Nono, an enemy of America . . .”:
  - $\text{Enemy}(\text{Nono}, \text{America})$

Colonel West domain in FOL

- Domain
  - Constants: Nono, West, America
  - Predicates:
    - Unary: American(x), Weapon(x), Missile(x), Hostile(x), Criminal(x)
    - Binary: Owns(x,y), Enemy(x,y)
    - Tertiary: Sells(x,y,z)
- KB

5. ". . . it is a crime for an American to sell weapons to hostile nations":
6. "Nono . . . has some missiles." :
7. "All of its missiles were sold to it by Colonel West":

- “. . . it is a crime for an American to sell weapons to hostile nations”:
  - $\forall xy\ z\ \text{American}(x) \land \text{Weapon}(y) \land \text{Sells}(x,y,z) \land \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$
- “Nono . . . has some missiles.”
  - $\exists x\ \text{Owns}(\text{Nono}, x) \land \text{Missile}(x)$
  - $\text{Owns}(\text{Nono}, M_1)$, $\text{Missile}(M_1)$: existential instantiation
- “All of its missiles were sold to it by Colonel West”:
  - $\forall x \text{Missile}(x) \land \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$

Forward chaining in PL

- AND-OR graph for forward chaining

Horn clauses

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

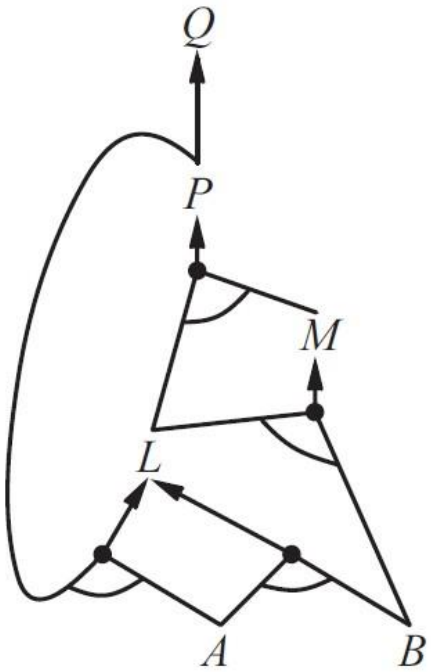
$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B

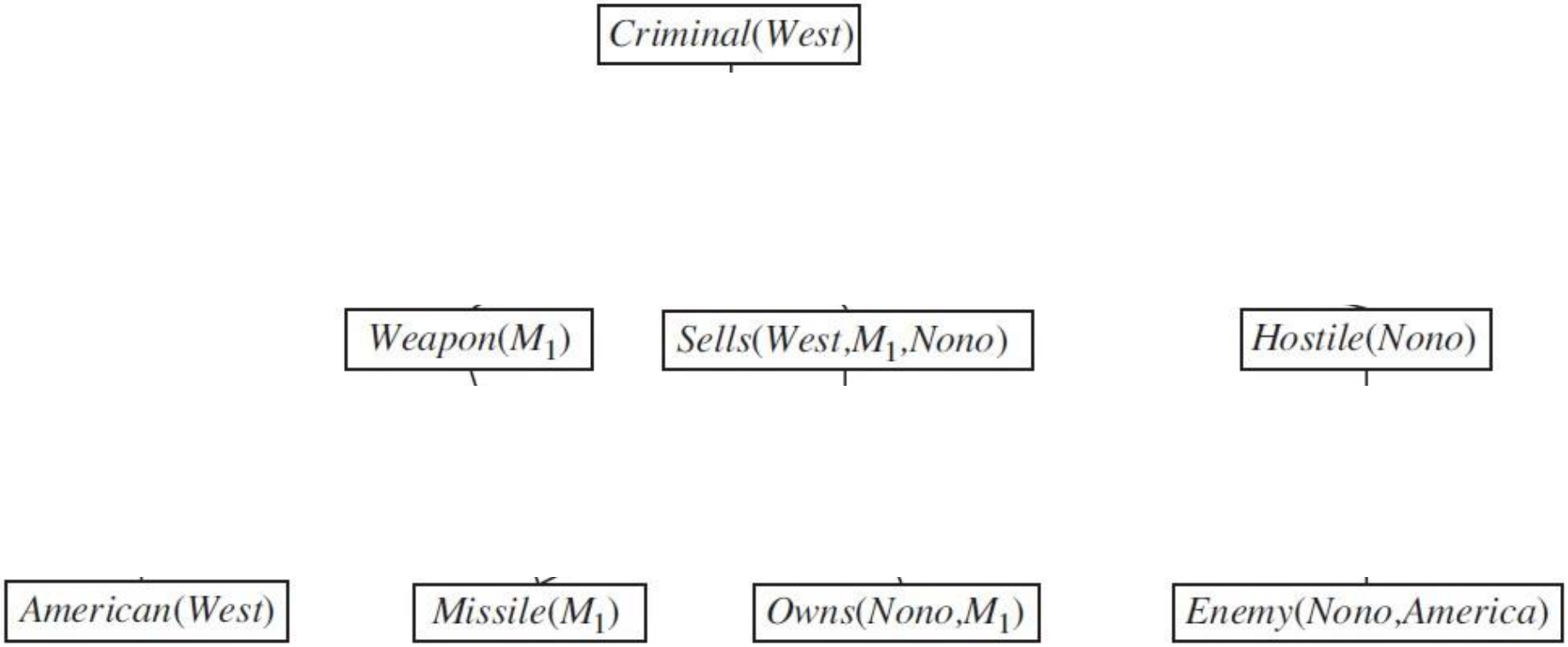
(a)

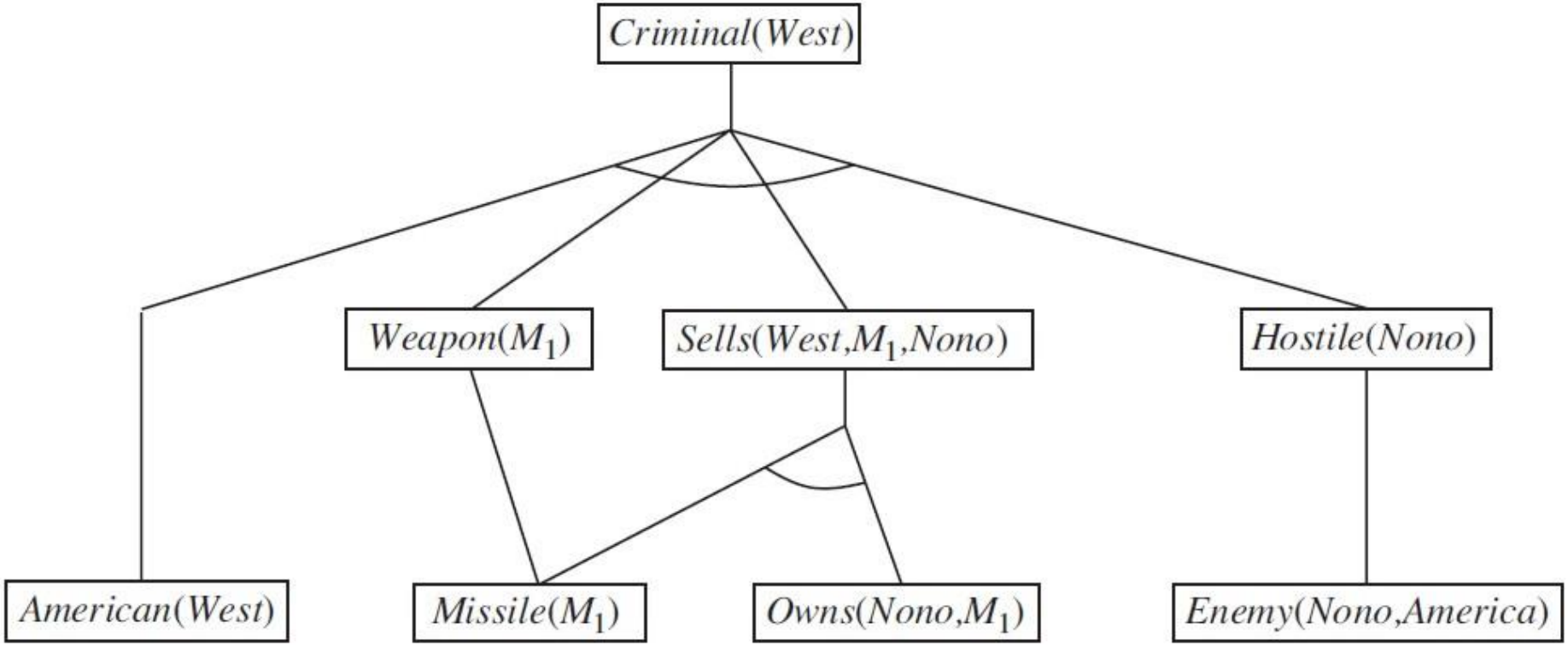


(b)

Forward chaining in FOL

- Starting from known facts, try to satisfy the premises by unification, and use Modus Ponens
- Simple facts
  - Owns(Nono, M_1), Missile(M_1), American(West), Enemy(Nono, America)
- Implications
  - $\forall xyz$ American(x) $\wedge$ Weapon(y) $\wedge$ Sells(x,y,z) $\wedge$ Hostile(z) $\Rightarrow$ Criminal(x)
  - $\forall x$ Missile(x) $\wedge$ Owns(Nono, x) $\Rightarrow$ Sells(West, x, Nono)
  - $\forall x$ Missile(x) $\Rightarrow$ Weapon(x)
  - $\forall x$ Enemy(x, America) $\Rightarrow$ Hostile(x)


Criminal(West)
Weapon(M_1)
Sells(West,M_1,Nono)
Hostile(Nono)
American(West)
Missile(M_1)
Owns(Nono,M_1)
Enemy(Nono,America)



- Cf. Backward chaining: start from *Criminal(West)*

Resolution

- Conjunctive normal form
  - American(x) ∧ Weapon(y) ∧ Sells(x,y,z) ∧ Hostile(z) ⇒ Criminal(x):
  - Missile(x) ∧ Owns(Nono, x) ⇒ Sells(West, x, Nono):
  - Missile(x) ⇒ Weapon(x):
  - Enemy(x, America) ⇒ Hostile(x):
  - Owns(Nono, M1), Missile(M1), American(West), Enemy(Nono, America)

Resolution

- Conjunctive normal form
  - American(x) ∧ Weapon(y) ∧ Sells(x,y,z) ∧ Hostile(z) ⇒ Criminal(x):
    - ~American(x) ∨ ~ Weapon(y) ∨ ~ Sells(x, y, z) ∨ ~Hostile(z) ∨ Criminal(x)
  - Missile(x) ∧ Owns(Nono, x) ⇒ Sells(West, x, Nono):
    - ~Missile(x) ∨ ~Owns(Nono, x) ∨ Sells(West, x, Nono)
  - Missile(x) ⇒ Weapon(x):
    - ~Missile(x) ∨ Weapon(x)
  - Enemy(x, America) ⇒ Hostile(x):
    - ~Enemy(x, America) ∨ Hostile(x)
  - Owns(Nono, M1), Missile(M1), American(West), Enemy(Nono, America)

- KB
  - ~American(x) ∨ ~ Weapon(y) ∨ ~ Sells(x, y, z) ∨ ~Hostile(z) ∨ Criminal(x)
  - ~Missile(x) ∨ ~Owns(Nono, x) ∨ Sells(West, x, Nono)
  - ~Missile(x) ∨ Weapon(x)
  - ~Enemy(x, America) ∨ Hostile(x)
  - Owns(Nono, M1), Missile(M1), American(West), Enemy(Nono, America)
- Query: Is Colonel West a criminal?
  - Add ~Criminal(West)
- Resolution in backward-chaining
  - Start by resolving ~Criminal(West)
  - Try to resolve the leftmost literal in the newly generated sentence

The crime example:

$\neg American(x) \vee \neg Weapon(y) \vee \neg Sells(x, y, z) \vee \neg Hostile(z) \vee Criminal(x)$

$\neg Missile(x) \vee \neg Owns(Nono, x) \vee Sells(West, x, Nono)$

$\neg Missile(x) \vee Weapon(x)$

$\neg Enemy(x, America) \vee Hostile(x)$

$Owns(Nono, M_1)$

$Missile(M_1)$

$American(West)$

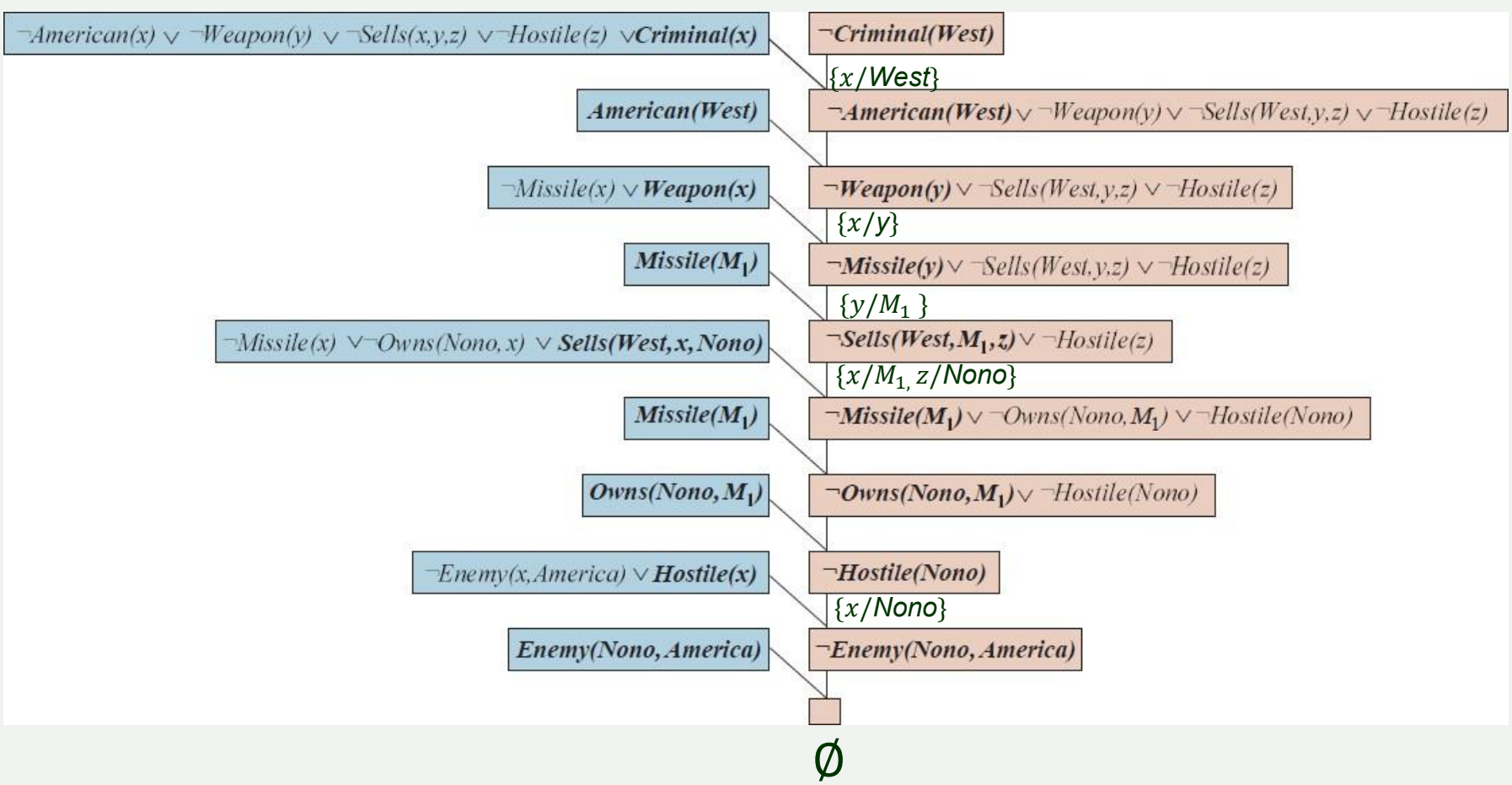
$Enemy(Nono, America)$

We prove *Criminal*(*West*) by adding

$\neg\ Criminal(West)$

and deriving the empty clause ∅.

Resolution by backward chaining in FOL



- Like backward chaining on the main spine.
- Always resolve with a clause containing a positive literal that unifies with the leftmost literal of the "current" clause on the spine.